

EPAM SDET / Test Automation Lead Interview Master Guide

Source questions + model answers + practical examples + senior-level follow-ups

Target profile: 13+ years | SDET / Test Automation Lead | Java | Selenium | Playwright | API
| CI/CD | Leadership

How to use this PDF

Use the bookmarks/contents to jump between domains. Each question has a concise interview answer and a practical example. The checkboxes are a preparation tracker. Questions marked as additions were added to cover gaps that are especially relevant to a senior SDET/lead profile.

Source basis

The uploaded document contains repeated EPAM interview-question sets covering Java, Selenium, API, TestNG/Cucumber, Jenkins/CI-CD, Git, SQL, Agile/testing, framework design, AI in testing and scenario-based troubleshooting. Repeated questions have been consolidated rather than copied multiple times.

Contents

1. 1. Java Core & Coding (19 questions)
2. 2. Selenium / Playwright Web Automation (14 questions)
3. 3. TestNG / Cucumber (7 questions)
4. 4. API / REST Assured (10 questions)
5. 5. Framework Architecture & Design (11 questions)
6. 6. CI/CD, Jenkins, Git & Maven (7 questions)
7. 7. SQL & Database (5 questions)
8. 8. Testing, Agile & Quality (10 questions)
9. 9. Leadership / Tech-Managerial (7 questions)
10. 10. AI in Testing & Your Current Profile (3 questions)
11. 11. Added High-Priority Playwright Questions (7 questions)
12. 12. Added Missing Cloud / DevOps / Leadership Depth (6 questions)

Priority for your profile: Framework architecture, Playwright migration, Java/Streams, API automation, CI/CD, flaky-test RCA, leadership scenarios and AI-assisted automation.

1. Java Core & Coding

Q1. Last non-repeating character in a string

Interview-ready answer: Use a frequency map first, then scan the string from right to left. The first character with frequency 1 is the last non-repeating character. This is $O(n)$ time and $O(k)$ space. Interview answer: "I avoid nested loops. I count characters once and scan again, which gives linear time complexity."

Practical example / code:

```
String input = "swiss";
Map<Character,Integer> freq = new LinkedHashMap<>();
for (char c : input.toCharArray())
    freq.put(c, freq.getOrDefault(c, 0) + 1);

for (int i = input.length() - 1; i >= 0; i--) {
    if (freq.get(input.charAt(i)) == 1) {
        System.out.println(input.charAt(i));
        break;
    }
}
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q2. Comparable vs Comparator

Interview-ready answer: Comparable defines a class's natural ordering through `compareTo()`. Comparator defines an external/custom ordering through `compare()`. Use Comparator when the same object needs multiple sort strategies. Example: Employee can naturally sort by id, while Comparator implementations sort by name or salary.

Practical example / code:

```
class Employee implements Comparable<Employee> {
    int id;
    String name;

    public int compareTo(Employee e) {
        return Integer.compare(this.id, e.id);
    }
}

Comparator<Employee> byName =
    Comparator.comparing(e -> e.name);
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q3. Java 8 features, Streams and Lambda expressions

Interview-ready answer: Key Java 8 features relevant to SDET work: lambda expressions, Stream API, functional interfaces, Optional, default/static interface methods, and the `java.time` API. Streams are useful for filtering test data, transforming API responses, extracting UI text and aggregating results.

Practical example / code:

```
List<String> titles = Arrays.asList("Zara", "Nike", "Apple");
List<String> result = titles.stream()
    .filter(s -> s.length() > 4)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q4. Meaning of :: in Java Streams

Interview-ready answer: The `::` operator is a method reference. It is shorthand for a lambda when an existing method already matches the required functional signature. `String::toUpperCase` is equivalent to `s -> s.toUpperCase()`.

Practical example / code:

```
List<String> upper = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q5. Where have you used Streams in your framework?

Interview-ready answer: Use a project-specific answer: filtering API collections, extracting product names from UI, validating expected vs actual values, transforming test data and aggregating results. Mention why: concise transformations without manual loops.

Practical example / code:

```
List<String> products = page.locator(".card-body b")
    .allTextContents();

List<String> zara = products.stream()
    .filter(p -> p.toLowerCase().contains("zara"))
    .collect(Collectors.toList());
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q6. Custom exception in Java

Interview-ready answer: Create a class extending Exception for a checked custom exception or RuntimeException for an unchecked one. Throw it with throw new ...; and handle or propagate it according to the layer's responsibility.

Practical example / code:

```
class LoginFailedException extends RuntimeException {
    public LoginFailedException(String message) {
        super(message);
    }
}

throw new LoginFailedException("Login failed for test user");
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q7. Serialization and deserialization

Interview-ready answer: Serialization converts an object into a byte representation; deserialization reconstructs the object. In API automation, JSON serialization/deserialization is more common and is often handled with Jackson ObjectMapper.

Practical example / code:

```
ObjectMapper mapper = new ObjectMapper();

String json = mapper.writeValueAsString(employee);
Employee e = mapper.readValue(json, Employee.class);
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q8. OOP concepts using a Savings Bank example

Interview-ready answer: Encapsulation: private balance with deposit/withdraw methods. Inheritance: SavingsAccount extends BankAccount. Abstraction: abstract account operations. Polymorphism: an Account reference can point to SavingsAccount or CurrentAccount and invoke overridden behavior.

Practical example / code:

```
abstract class BankAccount {
    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public abstract double calculateInterest();
}

class SavingsAccount extends BankAccount {
    public double calculateInterest() {
        return 0.04;
    }
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q9. Singleton design pattern

Interview-ready answer: Singleton ensures one shared instance. In test automation it can be useful for immutable configuration or a central configuration provider, but it should not be used to hold mutable WebDriver state because parallel tests can interfere with each other.

Practical example / code:

```
class ConfigManager {
    private static final ConfigManager INSTANCE =
        new ConfigManager();

    private ConfigManager() {}

    public static ConfigManager getInstance() {
        return INSTANCE;
    }
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q10. LinkedList use in automation

Interview-ready answer: LinkedList is useful when frequent insertion/removal at the ends is required. A practical example is maintaining a queue of test tasks or retry items. For random access, ArrayList is generally more appropriate.

Practical example / code:

```
Queue<String> retryQueue = new LinkedList<>();
retryQueue.add("LoginTest");
retryQueue.add("CheckoutTest");
String next = retryQueue.poll();
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q11. Java exception hierarchy and checked exceptions

Interview-ready answer: Throwable is the root, with Error and Exception beneath it. RuntimeException is an unchecked subclass of Exception. Checked exceptions are compiler-enforced, such as IOException and SQLException; they must be caught or declared.

Practical example / code:

```
try {
    Files.readAllLines(Paths.get("testdata.txt"));
} catch (IOException e) {
    // handle or propagate
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q12. Fail-fast vs fail-safe iterators

Interview-ready answer: Fail-fast iterators typically detect structural modification and throw `ConcurrentModificationException`. Fail-safe is an informal term commonly used for iterators over concurrent/copy-on-write collections that can iterate without failing because of the modification model.

Practical example / code:

```
List<String> list = new ArrayList<>(Arrays.asList("A","B"));
for (String s : list) {
    // list.add("C"); // may cause ConcurrentModificationException
}

CopyOnWriteArrayList<String> safe =
    new CopyOnWriteArrayList<>(list);
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q13. When to use ConcurrentHashMap

Interview-ready answer: Use it when multiple threads need concurrent access to a map and you need thread-safe operations with better concurrency than synchronizing an entire `HashMap`. It is useful for shared read-heavy caches or execution metadata in parallel test infrastructure.

Practical example / code:

```
ConcurrentHashMap<String,String> tokens =
    new ConcurrentHashMap<>();
tokens.put("qaUser", "token123");
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q14. == vs equals()

Interview-ready answer: For objects, `==` compares references; `equals()` compares logical equality when the class overrides it. `String` is the classic example. Always use `equals` for value comparison and implement `hashCode` consistently when overriding `equals`.

Practical example / code:

```
String a = new String("EPAM");
String b = new String("EPAM");

System.out.println(a == b);          // false
System.out.println(a.equals(b));     // true
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q15. NullPointerException handling

Interview-ready answer: Prevent it by validating inputs, using clear object lifecycle rules, avoiding unnecessary nulls, and using `Optional` where appropriate. Do not simply catch `NullPointerException` as normal control flow; find and fix the source.

Practical example / code:

```
Objects.requireNonNull(page, "page must not be null");
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q16. Collections: List vs Set and duplicate removal

Interview-ready answer: List preserves order and allows duplicates. Set represents unique values. Use LinkedHashSet when you need uniqueness while preserving insertion order.

Practical example / code:

```
List<String> names = Arrays.asList("A", "B", "A", "C");
Set<String> unique = new LinkedHashSet<>(names);
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q17. Second largest number in an array

Interview-ready answer: Track the largest and second-largest values in one pass, carefully handling duplicates. Clarify whether “second largest” means second distinct largest.

Practical example / code:

```
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int n : arr) {
    if (n > first) {
        second = first;
        first = n;
    } else if (n > second && n != first) {
        second = n;
    }
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q18. Next greater element

Interview-ready answer: For a single key, scan to the right until a greater value is found. For all elements, a monotonic stack gives $O(n)$ time instead of $O(n^2)$. Mention the optimisation if the interviewer asks for a better solution.

Practical example / code:

```
int nextGreater(int[] a, int key) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == key) {
            for (int j = i + 1; j < a.length; j++)
                if (a[j] > key) return a[j];
            break;
        }
    }
    return -1;
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q19. Sort a list without built-in sort

Interview-ready answer: Use an elementary algorithm such as insertion sort or selection sort if the interviewer specifically forbids library sorting. State the complexity. In production code, prefer the standard library unless the exercise requires otherwise.

Practical example / code:

```
for (int i = 1; i < a.length; i++) {
    int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j--];
    }
    a[j + 1] = key;
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

2. 2. Selenium / Playwright Web Automation

Q20. Selenium 4 new features

Interview-ready answer: Important Selenium 4 topics: W3C WebDriver standard alignment, relative locators, Selenium Grid 4 architecture and observability, improved DevTools integration, and modern browser capabilities. Give the feature list, then explain one you used rather than reciting names only.

Practical example / code:

```
WebElement element = driver.findElement(
    RelativeLocator.with(By.tagName("input"))
        .below(By.id("email"))
);
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q21. Page Object vs Page Factory

Interview-ready answer: Page Object Model is a design approach where page classes encapsulate locators and actions. PageFactory is a Selenium support mechanism historically used to initialise @FindBy fields. POM does not require PageFactory. Modern frameworks can use plain By locators or direct locator APIs and still be POM.

Practical example / code:

```
class LoginPage {
    private By user = By.id("userEmail");
    private By password = By.id("userPassword");

    void login(String u, String p) {
        driver.findElement(user).sendKeys(u);
        driver.findElement(password).sendKeys(p);
    }
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q22. Page Factory: what is it and disadvantages?

Interview-ready answer: PageFactory initializes fields annotated with @FindBy. Disadvantages include reliance on Selenium's proxy mechanism, less explicit lifecycle, and less value when using modern Selenium APIs or Playwright locators. For new frameworks, choose a style that makes waits and locator behaviour explicit.

Practical example / code:

```
@FindBy(id = "userEmail")
WebElement userEmail;
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q23. Dynamic web elements

Interview-ready answer: Prefer stable attributes such as data-testid, accessible roles or semantic attributes. Use parameterised locators and relative relationships rather than brittle absolute XPath. Avoid sleeps; synchronize on meaningful UI state.

Practical example / code:

```
By product = By.xpath(
    "//div[contains(@class, 'card-body')][.//b[normalize-space()='Zara Coat 3']]"
);
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q24. Implicit vs explicit vs fluent wait

Interview-ready answer: Implicit wait applies globally to element lookup. Explicit wait waits for a defined condition. Fluent wait is an explicit wait with configurable timeout, polling interval and ignored exceptions. Avoid mixing implicit and explicit waits because timing becomes harder to reason about.

Practical example / code:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.elementToBeClickable(
    By.id("login"))).click();
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q25. Shadow DOM handling

Interview-ready answer: For open shadow DOM, modern Selenium can locate shadow roots through `getShadowRoot()`, while Playwright automatically pierces open shadow DOM for most locators. Closed shadow roots cannot be directly traversed from outside; test through supported public UI or application hooks.

Practical example / code:

```
WebElement host = driver.findElement(By.cssSelector("my-component"));
SearchContext root = host.getShadowRoot();
root.findElement(By.cssSelector("#save")).click();
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q26. Multiple windows/tabs

Interview-ready answer: Capture the original handle, trigger the new window, obtain all handles and switch to the handle that is different from the original. In Playwright, use `context.waitForEvent('page')` for a new page.

Practical example / code:

```
String original = driver.getWindowHandle();
driver.findElement(By.id("newWindow")).click();

for (String h : driver.getWindowHandles()) {
    if (!h.equals(original)) {
        driver.switchTo().window(h);
        break;
    }
}
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q27. StaleElementReferenceException

Interview-ready answer: It occurs when a previously located element is no longer attached to the current DOM. Re-locate the element at the time of interaction, wait for the correct state, and avoid storing `WebElement` instances for long-lived dynamic pages.

Practical example / code:

```
By save = By.id("save");
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(save))
    .click();
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q28. Overlay blocks click

Interview-ready answer: Wait for the overlay to disappear and the target to become clickable. Inspect the DOM to identify whether the overlay is expected application behaviour. JavaScript click should be a last resort because it can bypass real user interaction.

Practical example / code:

```
wait.until(ExpectedConditions.invisibilityOfElementLocated(
    By.cssSelector(".loading-overlay")));
wait.until(ExpectedConditions.elementToBeClickable(button)).click();
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q29. Non-select dropdown

Interview-ready answer: Treat it as a normal interactive component: click the control, locate the visible options, and select by accessible text or stable attribute. Selenium Select works only for a real <select> element.

Practical example / code:

```
driver.findElement(By.id("country")).click();
driver.findElement(By.xpath("//li[normalize-space()='India']")).click();
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q30. File upload

Interview-ready answer: For an input type=file, send the absolute file path with sendKeys. In Playwright, use setInputFiles. Avoid OS-level automation when the application exposes a normal file input.

Practical example / code:

```
driver.findElement(By.cssSelector("input[type='file']"))
    .sendKeys("C:\\files\\resume.pdf");
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q31. Broken links in UI

Interview-ready answer: Collect href values, then issue HTTP requests and validate the response status. Exclude non-HTTP links such as mailto unless the requirement includes them. In parallel execution, ensure each result has isolated logging/reporting.

Practical example / code:

```
HttpClient client = HttpClient.newHttpClient();
// send GET/HEAD requests for collected hrefs and assert status < 400
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q32. Synchronisation for slow pages

Interview-ready answer: Prefer condition-based waits: URL, DOM state, element visibility, enabled state, network response or application-specific readiness. Do not use Thread.sleep as the primary synchronization mechanism.

Practical example / code:

```
await page.locator("#dashboard").waitFor({state: "visible"});
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q33. Parallel execution

Interview-ready answer: Parallelize independent tests at worker/thread level, isolate test data and browser context, avoid shared mutable state, and make reports/artifacts unique per worker. In TestNG use parallel and thread-count; in Playwright use workers/projects.

Practical example / code:

```
<suite name="suite" parallel="tests" thread-count="4">
```

☐ Prep status: ■ Know ■ Practise ■ Master

3. 3. TestNG / Cucumber

Q34. Run only regression tests in TestNG

Interview-ready answer: Use groups. Mark regression tests with `@Test(groups = "regression")` and run the group through `testng.xml` or command-line configuration. Keep the suite selection outside test code.

Practical example / code:

```
@Test(groups = {"regression"})
public void checkoutRegression() {}

<groups>
  <run><include name="regression"/></run>
</groups>
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q35. Cucumber DataTable without Scenario Outline

Interview-ready answer: Use `DataTable` when a step needs a structured set of values without parameterizing the whole scenario. Convert it to maps or lists in the step definition.

Practical example / code:

```
Given("the following users exist", (DataTable table) -> {
    List<Map<String,String>> rows = table.asMaps();
});
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q36. Parallel execution in Cucumber

Interview-ready answer: Use the framework's parallel execution support and ensure scenario state is isolated. Each scenario should get its own driver/context and test data. Avoid static mutable fields shared by scenarios.

Practical example / code:

```
// Configure runner/build tool for parallel scenarios.
// Keep WebDriver/context per scenario/thread.
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q37. Conditional vs directional hooks in Cucumber

Interview-ready answer: Conditional hooks run only for scenarios matching a tag condition, for example `@Before("@api")`. Directional hooks are ordered hooks using an explicit order so setup/teardown happens in a predictable sequence. Use tags for scope and order for sequencing.

Practical example / code:

```
@Before(value = "@smoke", order = 1)
public void smokeSetup() {}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q38. Cucumber tags and variables in step definitions

Interview-ready answer: Tags select scenarios/features. Variables can be captured with Cucumber expressions or regex parameters and passed directly into the step method.

Practical example / code:

```
@When("I search for {string}")
public void search(String product) {
    page.search(product);
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q39. IAnnotationTransformer in TestNG

Interview-ready answer: IAnnotationTransformer lets you modify TestNG annotations at runtime, commonly to apply retry analyzers or alter test metadata. A common method is transform(ITestAnnotation annotation, Class testClass, Constructor testConstructor, Method testMethod).

Practical example / code:

```
public void transform(
    ITestAnnotation annotation,
    Class testClass,
    Constructor testConstructor,
    Method testMethod) {

    annotation.setRetryAnalyzer(RetryAnalyzer.class);
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q40. Custom reporting: Extent and Allure

Interview-ready answer: Reporting should capture test status, steps, screenshots, logs and environment metadata. Extent is commonly programmatic; Allure can integrate through adapters/annotations and produces an HTML report from result files.

Practical example / code:

```
Allure.step("Validate product list");
Allure.addAttachment(
    "Screenshot", "image/png",
    new ByteArrayInputStream(bytes), ".png"
);
```

☐ Prep status: ■ Know ■ Practise ■ Master

4. 4. API / REST Assured

Q41. ObjectMapper in REST Assured

Interview-ready answer: ObjectMapper is a Jackson utility for converting Java objects to/from JSON. It is useful for POJO serialization and deserialization. REST Assured can also serialize POJOs when Jackson is available.

Practical example / code:

```
ObjectMapper mapper = new ObjectMapper();
String body = mapper.writeValueAsString(employee);
Employee e = mapper.readValue(json, Employee.class);
```



Prep status: ■ Know ■ Practise ■ Master

Q42. HTTP 405

Interview-ready answer: 405 Method Not Allowed means the server understands the HTTP method but the resource does not allow that method for the target endpoint. Check the endpoint contract, HTTP verb, URL, route, API version and allowed methods. Do not “fix” it by changing the method blindly; verify the API contract.

Practical example / code:

```
POST /users
// If the endpoint supports GET but not POST,
// a 405 can be returned. Verify API documentation.
```



Prep status: ■ Know ■ Practise ■ Master

Q43. REST Assured POST with POJO and Basic Auth

Interview-ready answer: Create a POJO for the request, serialize it through REST Assured/Jackson, send Basic Authentication, then validate status and response fields. Keep secrets in environment/CI variables rather than source code.

Practical example / code:

```
given()
    .auth().basic(user, password)
    .contentType(ContentType.JSON)
    .body(employee)
.when()
    .post("/employees")
.then()
    .statusCode(201);
```



Prep status: ■ Know ■ Practise ■ Master

Q44. JSON Schema validation

Interview-ready answer: Schema validation checks the response structure, data types and required properties against a JSON Schema. It complements field assertions; it does not replace business-rule validation.

Practical example / code:

```
given()
    .when().get("/employees/101")
    .then().assertThat()
    .body(matchesJsonSchemaInClasspath("employee-schema.json"));
```



Prep status: ■ Know ■ Practise ■ Master

Q45. Hamcrest Matchers

Interview-ready answer: Hamcrest provides readable matchers such as equalTo, hasItem, containsString and greaterThan. It makes assertions composable and expressive.

Practical example / code:

```
then().body("name", equalTo("Swapnil"))
      .body("age", greaterThan(18));
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q46. Token-based API authentication

Interview-ready answer: Obtain the token through the authentication endpoint or secure test setup, then pass it as a Bearer token to subsequent requests. Cache it only within the required test/session scope and avoid logging secrets.

Practical example / code:

```
given()
  .header("Authorization", "Bearer " + token)
.when()
  .get("/orders")
.then()
  .statusCode(200);
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q47. API negative testing

Interview-ready answer: Cover invalid authentication, missing mandatory fields, invalid formats, invalid IDs, unsupported methods, boundary values, authorisation failures and malformed payloads. Assert both status and meaningful error contract.

Practical example / code:

```
given().body("{}")
.when().post("/employees")
.then().statusCode(400)
      .body("error", notNullValue());
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q48. API chaining

Interview-ready answer: Extract a value from one response and use it in the next request. For example create user -> extract id -> get user -> update -> delete. Keep dependent data scoped to the scenario.

Practical example / code:

```
int id = given().body(employee)
  .post("/employees")
  .then().extract().path("id");

given().get("/employees/" + id)
  .then().statusCode(200);
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q49. API returns 200 but response is wrong

Interview-ready answer: Validate business content, schema, mandatory fields, data types, headers and relationships—not only status code. Compare against a trusted expected value or contract.

Practical example / code:

```
then()
  .statusCode(200)
  .body("status", equalTo("ACTIVE"))
  .body("id", notNullValue());
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q50. Store API response data dynamically

Interview-ready answer: Extract required fields into local variables or a scenario/test context. For reusable workflows, use a typed response POJO. Avoid static global state because it breaks parallel execution.

[Practical example / code:](#)

```
String orderId = response.jsonPath().getString("orderId");
```



Prep status: ■ Know ■ Practise ■ Master

5. 5. Framework Architecture & Design

Q51. Explain your automation framework architecture

Interview-ready answer: For your interview, describe a layered framework: tests/specs -> business/page/API objects -> reusable utilities -> configuration/test data -> reporting -> CI/CD. Explain your real migration from Selenium to Playwright, selector strategy, reusable Browser/Assert utilities, API/UI coverage and CI execution.

[Practical example / code:](#)

```
Example architecture:
tests/
pages/
api/
utils/
resources/selectors/
test-data/
config/
reports/
.github/ or Jenkinsfile
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q52. Framework from scratch

Interview-ready answer: Start with requirements and supported layers, choose language/tooling, establish project structure, config management, logging/reporting, reusable driver/browser lifecycle, page/API layers, test data, assertions, parallelism, CI integration and quality gates. Add conventions and documentation before scaling.

[Practical example / code:](#)

```
A good bootstrap sequence:
1. Build
2. Config
3. Browser/API clients
4. POM
5. Assertions
6. Reporting
7. Parallelism
8. CI
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q53. Design patterns used in automation

Interview-ready answer: Common patterns: POM for UI abstraction, Factory for creating browser/client variants, Builder for complex test data, Singleton only for safe immutable configuration, Strategy for interchangeable behaviours, and dependency injection/fixtures for lifecycle management.

[Practical example / code:](#)

```
interface BrowserFactory {
    Browser create();
}
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q54. Encapsulation in POM

Interview-ready answer: Keep locators and implementation details private to page objects and expose business-oriented methods. Tests should say login(), searchProduct() or checkout(), not manipulate low-level selectors everywhere.

Practical example / code:

```
class LoginPage {
    private By user = By.id("userEmail");

    public void login(String email, String password) {
        // locator details remain inside the page object
    }
}
```

☐

Prep status: ■ Know ■ Practise ■ Master

Q55. Hybrid framework design

Interview-ready answer: Combine complementary approaches: POM for UI, data-driven inputs, API clients, reusable utilities, tagging and configuration. The goal is maintainability and coverage, not combining patterns simply for their names.

Practical example / code:

```
UI test -> Page Object
API test -> API Client
Data -> JSON/CSV/DB
Config -> environment variables
CI -> Jenkins/GitHub Actions
```

☐

Prep status: ■ Know ■ Practise ■ Master

Q56. Test data management

Interview-ready answer: Separate static reference data from generated data. Use environment-specific configuration, factories/builders for dynamic data, API setup where possible, cleanup, unique identifiers and secrets from CI secret stores.

Practical example / code:

```
const user = {
    email: `qa_${Date.now()}@example.com`
};
```

☐

Prep status: ■ Know ■ Practise ■ Master

Q57. Retry logic for flaky tests

Interview-ready answer: Retry is a safety net, not a fix. First identify root cause. Use limited retries only for known transient infrastructure failures and report retries separately. Never hide product defects by retrying indefinitely.

Practical example / code:

```
// Playwright config
retries: process.env.CI ? 2 : 0
```

☐

Prep status: ■ Know ■ Practise ■ Master

Q58. Framework execution is slow

Interview-ready answer: Measure first. Typical improvements: parallel workers, API setup instead of UI setup, remove fixed sleeps, reuse authentication state where safe, reduce redundant setup, run targeted suites, optimise data creation and use sharding for large suites.

Practical example / code:

```
// Prefer API setup:
await request.post("/test-data/users");
// Then start UI test already authenticated where supported.
```

☐

Prep status: ■ Know ■ Practise ■ Master

Q59. Cross-browser testing

Interview-ready answer: Define browser matrix based on supported products and risk. Run smoke on every supported browser and broader regression according to release needs. Use Playwright projects or Selenium Grid/cloud platforms for parallel coverage.

[Practical example / code:](#)

```
projects: [  
  { name: "chromium", use: { ...devices["Desktop Chrome"] } },  
  { name: "firefox", use: { ...devices["Desktop Firefox"] } }  
]
```



Prep status: ☒ Know ☒ Practise ☒ Master

Q60. Parallel reports/screenshots collision

Interview-ready answer: Use worker/test identifiers in artifact names, framework-managed test result directories, and avoid global filenames. Each test should own its context and artifact path.

[Practical example / code:](#)

```
const file = `artifacts/${testInfo.testId}/screenshot.png`;
```



Prep status: ☒ Know ☒ Practise ☒ Master

Q61. Selenium-to-Playwright migration

Interview-ready answer: Inventory existing coverage, identify high-value tests, map locator strategy, convert setup and waits, redesign POM around Playwright locators, migrate API/utilities where useful, introduce tracing/screenshots, run old/new suites in parallel for confidence, then retire redundant Selenium code. In your answer, mention your real selector JSON/data-testid and removal of hard waits.

[Practical example / code:](#)

```
await page.getByTestId("login-button").click();  
await expect(page.getByRole("heading", {name: "Dashboard"}))  
  .toBeVisible();
```



Prep status: ☒ Know ☒ Practise ☒ Master

6. 6. CI/CD, Jenkins, Git & Maven

Q62. Jenkins process to run Selenium/Playwright tests

Interview-ready answer: Typical flow: checkout -> install dependencies -> build -> run tagged/smoke/regression tests -> collect results -> publish report -> archive artifacts -> notify. For parallelism, use Jenkins stages/agents or framework workers. Keep credentials in Jenkins credentials/secrets.

Practical example / code:

```
pipeline {
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Test') { steps { sh 'mvn test' } }
    stage('Report') { steps { junit '**/surefire-reports/*.xml' } }
  }
}
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q63. Continuous Integration vs Continuous Delivery/Deployment

Interview-ready answer: CI integrates and validates changes frequently. Continuous Delivery keeps software in a releasable state and generally requires a deployment decision/approval. Continuous Deployment automatically deploys validated changes to the target environment.

Practical example / code:

```
Commit -> Build -> Test -> Package -> Deploy to test
-> approval (delivery) -> production
or automatic production deployment (deployment).
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q64. Git cherry-pick

Interview-ready answer: Cherry-pick applies the changes introduced by a specific commit onto the current branch. Use it when you need one targeted fix without merging the entire source branch. Resolve conflicts, test, and push the resulting commit.

Practical example / code:

```
git fetch origin
git checkout release
git cherry-pick <commit-sha>
# resolve conflicts if required
git cherry-pick --continue
git push origin release
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q65. Git branching strategy

Interview-ready answer: Choose a strategy appropriate to release cadence: trunk-based development for frequent integration or a feature/release/hotfix model where release isolation is required. Protect main branches, use pull requests, reviews and automated checks.

Practical example / code:

```
main
  ■■■ feature/login
  ■■■ feature/payment
  ■■■ hotfix/security-fix
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q66. git fetch vs pull; reset vs revert

Interview-ready answer: fetch downloads remote references without changing your working branch. pull generally fetches and integrates the remote changes. reset moves the current branch pointer and can rewrite local history; revert creates a new commit that undoes an earlier commit and is safer for shared history.

[Practical example / code:](#)

```
git fetch origin
git pull --rebase origin main

git reset --hard HEAD~1    # local history rewrite
git revert <sha>           # shared-history friendly
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q67. Maven profiles and plugins

Interview-ready answer: Plugins provide build/test functionality such as compiler, Surefire and Failsafe. Profiles allow environment- or purpose-specific configuration, for example smoke vs regression or QA vs staging.

[Practical example / code:](#)

```
mvn test -Pregression
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q68. TestNG parallelism

Interview-ready answer: Configure parallel mode and thread count in testng.xml or programmatically. Ensure driver/browser instances, test data and reports are thread-safe and not stored in unsafe static fields.

[Practical example / code:](#)

```
<suite parallel="methods" thread-count="4">
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

7. 7. SQL & Database

Q69. DDL vs DML

Interview-ready answer: DDL defines database structure: CREATE, ALTER, DROP, TRUNCATE. DML changes row data: INSERT, UPDATE, DELETE. SELECT is commonly classified as DQL.

[Practical example / code:](#)

```
CREATE TABLE employee(id INT);
INSERT INTO employee VALUES (1);
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q70. SQL subqueries

Interview-ready answer: A subquery is a query nested inside another query. It can appear in WHERE, FROM or SELECT depending on the use case. Correlated subqueries depend on the outer row and can be more expensive.

[Practical example / code:](#)

```
SELECT name
FROM employee
WHERE salary > (
    SELECT AVG(salary) FROM employee
);
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q71. SQL joins

Interview-ready answer: INNER JOIN returns matching rows. LEFT JOIN keeps all rows from the left table. RIGHT JOIN keeps all rows from the right. FULL OUTER JOIN returns all rows where supported. Explain joins using business entities such as employee and department.

[Practical example / code:](#)

```
SELECT e.name, d.name
FROM employee e
JOIN department d
    ON e.department_id = d.id;
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q72. Second highest salary

Interview-ready answer: If duplicate salaries should count as one rank, use DENSE_RANK or a distinct subquery. Clarify the requirement before coding.

[Practical example / code:](#)

```
SELECT MAX(salary)
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

☐ Prep status: ☒ Know ☒ Practise ☒ Master

Q73. Database validation in automation

Interview-ready answer: Validate data written by the UI/API against the database where the database is part of the test contract. Use a DB utility or service layer, parameterized queries, read-only credentials where possible, cleanup, and avoid coupling every UI test directly to implementation tables.

Practical example / code:

```
// Conceptual flow:  
// create via API -> query DB -> assert persisted status  
SELECT status FROM orders WHERE order_id = ?;
```



Prep status: ■ Know ■ Practise ■ Master

8. 8. Testing, Agile & Quality

Q74. Test Pyramid

Interview-ready answer: The pyramid recommends more fast, focused unit/component tests, fewer service/API tests, and fewer slow end-to-end UI tests. Choose automation level based on feedback speed, stability and defect location.

[Practical example / code:](#)

```
Unit/component: many
API/service: medium
UI E2E: fewer, high-value journeys
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q75. Test Plan

Interview-ready answer: A Test Plan describes what will be tested and how the project will execute testing: scope, objectives, strategy/approach, environments, data, roles, schedule, entry/exit criteria, risks, dependencies and reporting.

[Practical example / code:](#)

```
Sections:
Scope | Approach | Environment | Data
Schedule | Roles | Risks | Entry/Exit criteria
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q76. Test Plan vs Test Strategy

Interview-ready answer: A strategy is the higher-level approach to quality/testing for a product or organisation. A test plan is more execution-oriented for a project/release and describes scope, schedule, resources, environments and deliverables. Terminology varies by organisation.

[Practical example / code:](#)

```
Strategy: "How we approach quality."
Plan: "What this release/team will execute, when and with whom."
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q77. Test design techniques

Interview-ready answer: Use equivalence partitioning, boundary value analysis, decision tables, state transition, pairwise/combinatorial testing and exploratory techniques as appropriate. Choose based on risk and input complexity.

[Practical example / code:](#)

```
Age field:
Boundary-focused tests:
- below minimum
- minimum
- just above minimum
- just below maximum
- maximum
- above maximum
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q78. Defect leakage

Interview-ready answer: Defect leakage measures defects escaping a test stage into a later stage, often production. A common project metric is production defects divided by total defects discovered across the defined lifecycle, multiplied by 100. Agree on the organisation's exact formula and population first.

Practical example / code:

```
Leakage % = Production defects /  
            (Pre-production defects + Production defects) * 100
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q79. RCA techniques

Interview-ready answer: Useful techniques include 5 Whys, Fishbone/Ishikawa, Pareto analysis and timeline/log correlation. For automation, reproduce, isolate, collect logs/traces, identify contributing conditions, fix the cause, and add a regression guard.

Practical example / code:

```
Failure -> classify -> reproduce -> collect evidence  
-> identify root cause -> corrective action  
-> regression test -> monitor
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q80. Agile estimation

Interview-ready answer: Common techniques include Planning Poker/story points, affinity estimation, analogous estimation and capacity-based planning. Separate complexity/uncertainty from raw hours and calibrate estimates using historical team data.

Practical example / code:

```
Story points:  
1, 2, 3, 5, 8, 13  
Discuss complexity, dependencies and uncertainty.
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q81. Agile metrics

Interview-ready answer: Examples: sprint predictability, cycle/lead time, escaped defects, defect trend, automation pass rate, flaky test rate, test execution duration, blocked work and deployment frequency. Avoid using a metric as a simplistic individual-performance score.

Practical example / code:

```
Track:  
- escaped defects  
- flaky tests  
- regression duration  
- automation coverage  
- cycle time
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q82. Manual test case example

Interview-ready answer: Use a simple login example and demonstrate positive, negative, boundary and security-oriented cases. Explain expected result, preconditions, data and traceability.

Practical example / code:

```
TC-LOGIN-01  
Precondition: active user  
Steps: enter valid email/password, click Login  
Expected: dashboard displayed
```

```
TC-LOGIN-02  
Invalid password -> validation/error shown
```

☐ Prep status: ■ Know ■ Practise ■ Master

Q83. Developer rejects a bug

Interview-ready answer: Stay evidence-based. Reproduce, provide steps, expected vs actual, logs/screenshots/video, environment/build and business requirement. Discuss directly, involve PO/BA when expected behaviour is ambiguous, and document the agreed decision rather than turning it into a personal conflict.

Practical example / code:

Evidence -> discussion -> requirement clarification
-> severity/priority agreement -> documented outcome



Prep status: ■ Know ■ Practise ■ Master

9. 9. Leadership / Tech-Managerial

Q84. Tell me about yourself

Interview-ready answer: Use a 60–90 second structure: 13+ years in quality engineering -> automation leadership -> web/API/mobile -> current Playwright migration and CI/CD -> healthcare project scale -> leadership/mentoring -> what you are looking to contribute next.

Practical example / code:

Structure:
Experience → current ownership → key achievements
→ leadership → technology → target role

☐ Prep status: ■ Know ■ Practise ■ Master

Q85. How do you handle a difficult team member?

Interview-ready answer: Start with facts and private discussion. Understand the cause, align expectations, agree measurable actions, provide support, follow up, and escalate only when necessary. Focus on behaviour and delivery impact, not personality labels.

Practical example / code:

Situation → private conversation → expectations
→ support → measurable follow-up → escalation if needed

☐ Prep status: ■ Know ■ Practise ■ Master

Q86. How do you handle disagreement with a developer/product owner?

Interview-ready answer: Use evidence and shared acceptance criteria. Separate technical facts from preference. If ambiguity remains, ask the product owner/business owner to clarify expected behaviour and document the decision.

Practical example / code:

Observed behaviour → requirement → impact → options
→ decision owner → documented outcome

☐ Prep status: ■ Know ■ Practise ■ Master

Q87. How do you decide automation candidates?

Interview-ready answer: Prioritise high-value, repeatable, stable and deterministic flows with strong regression value. Consider business risk, execution frequency, maintenance cost and test data complexity. Do not automate simply because a test is manual.

Practical example / code:

High value + repeatability + stability
+ regression frequency > maintenance cost

☐ Prep status: ■ Know ■ Practise ■ Master

Q88. Challenging automation issue you solved

Interview-ready answer: Use STAR. A strong example for you is the Selenium-to-Playwright migration: situation—flaky/slow legacy UI automation; task—improve reliability and execution; actions—Playwright, selector strategy, reusable Browser/Assert utilities, hard-wait removal, CI integration; result—describe only metrics you can substantiate.

Practical example / code:

STAR:
Situation → Task → Actions → Result
Add measurable evidence where available.

☐ Prep status: ■ Know ■ Practise ■ Master

Q89. Mentoring and technical leadership

Interview-ready answer: Explain how you establish coding standards, review PRs, pair on difficult problems, create reusable utilities, share knowledge, and gradually increase ownership in team members. Show that leadership includes enabling others, not just doing the hardest technical work yourself.

[Practical example / code:](#)

Standards → pairing → reviews → documentation
→ ownership → feedback

☐ Prep status: ■ Know ■ Practise ■ Master

Q90. Release-quality decision

Interview-ready answer: Use risk-based evidence: critical defects, regression status, impacted areas, test coverage, known limitations, environment confidence and business impact. Present the risk and options to the release decision owner rather than making unsupported promises.

[Practical example / code:](#)

Quality signal → risk → impact → mitigation
→ recommendation/options → decision owner

☐ Prep status: ■ Know ■ Practise ■ Master

10. 10. AI in Testing & Your Current Profile

Q91. How are you using AI day-to-day?

Interview-ready answer: Explain practical, controlled usage: AI-assisted test generation/refactoring, debugging, documentation and code review support. For your profile, mention Cursor and Claude with repository rules/skills rather than ad-hoc prompting. Keep proprietary data protected and review generated code before committing.

[Practical example / code:](#)

Workflow:
Understand task → provide bounded context/rules
→ generate/draft → review → run tests/static checks
→ refine → commit

☐ Prep status: ■ Know ■ Practise ■ Master

Q92. How would you use AI for test automation?

Interview-ready answer: Use AI for candidate test generation, locator suggestions, data generation, failure clustering, log summarisation, API contract assistance and documentation. Keep deterministic assertions and human review in the engineering workflow.

[Practical example / code:](#)

Failure logs → AI summary → probable patterns
→ engineer verifies root cause → regression fix

☐ Prep status: ■ Know ■ Practise ■ Master

Q93. AI risks in testing

Interview-ready answer: Risks include hallucinated code, false confidence, sensitive-data exposure, biased or incomplete test generation, and overfitting to current implementation. Mitigate with repository rules, approved data, review, deterministic execution and CI validation.

[Practical example / code:](#)

AI output ≠ evidence.
Generated test → review → execute → validate

☐ Prep status: ■ Know ■ Practise ■ Master

11. 11. Added High-Priority Playwright Questions

ADDED FOR YOUR PREPARATION — These questions fill gaps not sufficiently represented in the uploaded question set and are tailored to a senior SDET/Test Automation Lead profile.

Q94. Why Playwright instead of Selenium?

Interview-ready answer: Playwright provides browser automation with auto-waiting, isolated browser contexts, built-in tracing/screenshots, strong multi-page/frame handling and APIRequest support. Selenium remains valuable for broad WebDriver/Grid ecosystems. Give a project-specific reason rather than claiming one tool is universally better.

Practical example / code:

```
const context = await browser.newContext();
const page = await context.newPage();

await page.getByRole("button", {name: "Login"}).click();
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q95. Playwright locators and auto-waiting

Interview-ready answer: Prefer role, label, test id and text-based locators according to the application's accessibility/stability. Playwright waits for actionability before actions and supports web-first assertions. Avoid hard waits such as `waitForTimeout` for normal synchronization.

Practical example / code:

```
await page.getByLabel("Email").fill("qa@example.com");
await expect(page.getByRole("heading", {
  name: "Dashboard"
})).toBeVisible();
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q96. Playwright fixtures

Interview-ready answer: Fixtures provide controlled setup/teardown and dependency injection for tests. Built-in page/browser fixtures isolate browser state; custom fixtures can provide logged-in users, API clients or page objects.

Practical example / code:

```
exports.test = base.extend({
  loginPage: async ({page}, use) => {
    await use(new LoginPage(page));
  }
});
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q97. Playwright dialogs and frames

Interview-ready answer: Register a dialog handler before triggering the dialog. For frames, use `frameLocator` for stable locator-based interaction. For popups, wait for the page event while triggering the action.

Practical example / code:

```
page.on("dialog", async dialog => {
  console.log(dialog.message());
  await dialog.accept();
});

const frame = page.frameLocator("#course-iframe");
await frame.getByText("Lifetime Access").click();
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q98. Playwright trace and debugging

Interview-ready answer: Use trace recording on failure or during investigation. Inspect action snapshots, network, console and timing. Combine with screenshots/video where appropriate. The goal is evidence, not just a red test.

[Practical example / code:](#)

```
use: {  
  trace: "retain-on-failure",  
  screenshot: "only-on-failure"  
}
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q99. Playwright authentication state

Interview-ready answer: Authenticate once and reuse storage state when the application allows it. Keep credentials out of source control and regenerate state safely. Ensure tests that mutate user data use isolated accounts/data.

[Practical example / code:](#)

```
await context.storageState({  
  path: "playwright/.auth/user.json"  
});
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q100. API + UI test strategy

Interview-ready answer: Use API calls to set up data and validate backend behaviour quickly; use UI tests for critical user journeys and integration confidence. This reduces expensive UI setup while preserving end-to-end coverage.

[Practical example / code:](#)

```
API creates order  
→ UI opens order  
→ UI verifies user-visible status  
→ API verifies persisted state
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

12. 12. Added Missing Cloud / DevOps / Leadership Depth

ADDED FOR YOUR PREPARATION — These questions fill gaps not sufficiently represented in the uploaded question set and are tailored to a senior SDET/Test Automation Lead profile.

Q101. How would you design a scalable CI test pipeline?

Interview-ready answer: Split fast checks from full regression, parallelize workers, cache dependencies safely, publish JUnit/Allure artifacts, preserve traces/screenshots for failures, use environment-specific secrets, and make the pipeline fail for meaningful quality gates. Add scheduled full regression if needed.

[Practical example / code:](#)

```
PR: lint + unit + smoke
Nightly: full regression
Release: risk-based full suite + reports
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q102. How do you troubleshoot a test that passes locally but fails in Jenkins?

Interview-ready answer: Compare browser/version, OS/container, environment variables, secrets, base URL, test data, timing, parallelism and file-system paths. Inspect Jenkins console logs, Playwright trace, screenshots and network failures. Reproduce using the same container/agent image if possible.

[Practical example / code:](#)

```
Local vs CI matrix:
code | dependencies | browser | env | data
| timing | permissions | parallelism
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q103. How would you reduce a six-hour suite?

Interview-ready answer: Measure where time is spent, then remove redundant UI setup, parallelize independent tests, shard by suite, use API setup, reuse safe authentication state, eliminate hard waits, and quarantine only proven infrastructure failures. Validate that parallelism does not create data collisions.

[Practical example / code:](#)

```
6h serial
→ profile
→ parallel workers/shards
→ API setup
→ targeted suites
→ measure again
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q104. API intermittently returns HTTP 500

Interview-ready answer: Capture request/response, correlation ID, timestamp, environment and payload. Determine whether failures correlate with load, specific data, dependency outages or timeouts. Reproduce with repeated controlled calls, inspect service logs/metrics if available, and distinguish application defects from infrastructure/transient failures.

[Practical example / code:](#)

```
Client evidence + server correlation
→ reproduce
→ dependency analysis
→ root cause
→ regression/monitoring
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q105. Automation breaks after every UI change

Interview-ready answer: Move away from implementation-heavy selectors, use stable test IDs/accessibility contracts, centralise selectors, keep page methods business-oriented, and coordinate selector/testability contracts with developers. Add component-level tests where suitable.

[Practical example / code:](#)

```
Prefer:  
data-testid / role / label  
over brittle absolute XPath
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Q106. Microservices testing approach

Interview-ready answer: Test service contracts and APIs independently, use consumer/provider contract tests where appropriate, validate events/messages, use controlled test data and observability, and keep a smaller set of true end-to-end flows. Test failure handling and dependency timeouts as well.

[Practical example / code:](#)

```
Service A → API  
Service B → contract  
Event → message validation  
E2E → critical business journey
```

☐ Prep status: ☐ Know ☐ Practise ☐ Master

Final Rapid-Fire Revision

POM: Separates page/UI implementation details from tests; tests call business-level methods.

Page Factory: Selenium field-initialisation mechanism using `@FindBy`; not the same thing as POM.

Stream ::: Method reference shorthand.

405: HTTP method not allowed for the resource.

ConcurrentHashMap: Concurrent thread-safe map for shared mutable map access.

Flaky test: Investigate root cause first; common causes are synchronization, shared state, data collisions and environment instability.

Test Pyramid: Many fast unit/component tests, fewer service/API tests, fewer expensive UI E2E tests.

Cherry-pick: Apply one commit's changes onto another branch.

fetch vs pull: fetch downloads remote refs; pull fetches and integrates.

reset vs revert: reset moves branch history; revert creates a new undo commit.

API 200 but wrong data: Validate schema, fields, business rules and relationships—not just status.

Playwright advantage: Auto-waiting, isolated contexts, tracing, multi-page support and integrated API capabilities.

Final interview rule

For senior-level questions, answer in this order: **concept** → **why** → **how you implemented it** → **trade-off** → **real project example** → **measurable result**. Do not invent metrics. If you do not have an exact number, describe the observable improvement honestly.

Source note: This guide consolidates and expands the questions in the uploaded "EPAM interview Question.docx"; the source contains repeated question sets across 10 pages. Source-derived topics include the Round 1/2 lists and the later SDET/framework/API/AI sections.